SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Smart IT Asset Lifecycle & Security Compliance Management System

Index Number: DEH-IT-2324-F-0057

Author: W.G.C.D. Ravihara

Document Version: 1.0.0

Date: July 5, 2026

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Purpose

This document explains what the Smart IT Asset Lifecycle & Security Compliance Management System needs to do, and how well it needs to do it. It is written for the people who will build, test, or evaluate the system: developers, project supervisors, and academic reviewers of this HNDIT individual project.

The aim is to give everyone the same clear picture of the system before development starts, so there is no confusion later about what the finished product should include.

1.2 Scope

The system gives an organization one central place to manage its IT hardware and keep an eye on its security. In simple terms, it covers five main jobs:

1. Asset Inventory Control: Keeping a full record of every device, including where it is, when it was bought, and its maintenance history.
2. Asset Assignment: Tracking which employee is using which device, and logging when a device is returned or moved to someone else.
3. Security Monitoring Agent: A small background program on each Windows computer that reports hardware usage and security settings.
4. Compliance Scoring: Automatically working out a safety score for each device based on its security settings.
5. Remote Control: Letting an administrator restart or shut down a device straight from the dashboard.

1.3 Definitions, Acronyms and Abbreviations

- EDR — Endpoint Detection and Response, the background monitoring agent.
- SRS — Software Requirements Specification (this document).
- JWT — JSON Web Token, used to keep user logins secure.
- RBAC — Role-Based Access Control, which decides what each user is allowed to see or do.

- API — Application Programming Interface, the way the frontend and backend talk to each other.
- WS / WSS — WebSocket / WebSocket Secure, used for live, real-time updates.
- AV / FW — Antivirus / Firewall.
- USBSTOR — The Windows registry setting that controls whether USB storage devices are allowed.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- PostgreSQL 16 Global Development Documentation.
- React.js and Socket.io developer guidelines.

2. OVERALL DESCRIPTION

2.1 Product Perspective

The system is a self-contained platform made up of three connected parts, shown in Figure 2.1 below.

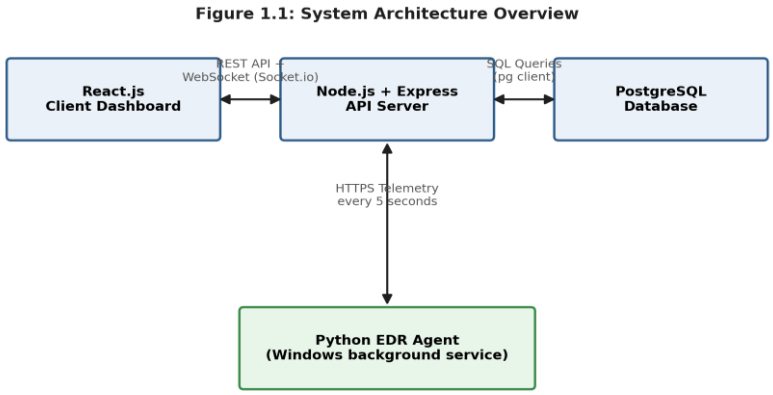

Figure 2.1: System Architecture Overview

- Agent (Python program): Installed on each target computer. It reads local hardware and security details and waits for any remote commands.
- API Server (Node.js and Express): Sits in the middle. It saves data to the database, works out each device's risk score, and keeps a queue of pending commands.
- Dashboard (React website): What the administrator or viewer sees in their browser — device lists, compliance scores, and live charts.

2.2 Product Functions

- Secure login for administrators and viewers.
- Registering new IT hardware and recording its specifications.
- Assigning assets to departments or specific employees.
- Watching live CPU, RAM, disk, and network stats through a WebSocket connection.
- Checking Windows security settings such as Windows Defender, the firewall, USB restrictions, and password age rules.

- Warning the admin when unapproved software is detected running on a device.
- Sending remote shutdown or restart commands to a device.
- Safely clearing old logs without deleting the record of the agent that is currently active.

2.3 User Classes and Characteristics

- Viewer User: Can only look at asset records, compliance reports, and live graphs. Cannot change anything or send commands.
- System Administrator: Can add, edit, or remove assets, assign devices to employees, and send remote commands.
- Super Administrator: Has every admin permission, plus the ability to clear telemetry logs and change server-level settings.

2.4 Operating Environment

- Backend Server: Node.js (version 18 or newer), running on Windows Server or Linux.
- Database: PostgreSQL (version 14 or newer).
- Dashboard Browser: Any modern browser that supports WebSockets, such as Chrome 90+, Firefox 88+, or Edge 90+.
- Client Agent: Windows 10 or 11, with Python 3.10+ installed, or the packaged standalone .exe version.

2.5 Design Constraints

- Remote restart and shutdown commands are built for Windows machines using PowerShell, so other operating systems are not fully supported yet.
- The WebSocket connection needs a steady network link between each device and the backend server.
- Port 5000 (backend) and Port 5173 (frontend) must stay open on the network for the system to work correctly.

3. FUNCTIONAL REQUIREMENTS

This chapter lists what the system must be able to do. Each requirement has a short ID so it can be tracked and tested easily.

3.1 User Login and Access Control

- FR-1.1: The system must check a user's username and password against the securely stored (hashed) values in the database.
- FR-1.2: Once login succeeds, the backend must issue a signed JWT token to keep the session secure.
- FR-1.3: Menu items in the sidebar must show or hide automatically depending on whether the user is a viewer, admin, or super admin.

3.2 Asset Registry and Inventory Control

- FR-2.1: Users must be able to add a new asset with details such as ID, category, name, serial number, cost, and status.
- FR-2.2: The system must not allow two assets to share the same serial number.
- FR-2.3: Users must be able to filter the asset list by category (laptop, server, workstation) or by status (active, under repair, retired).

3.3 Employee Assignment and Return Tracking

- FR-3.1: Admins must be able to assign an available asset to a registered employee using their email or employee ID.
- FR-3.2: The system must record the date of every assignment, so a full history is kept.
- FR-3.3: When an asset is returned or transferred, its status must update automatically in the main asset table.

3.4 Live Telemetry Logging and Streaming

- FR-4.1: The background agent must collect CPU, RAM, disk, and network usage figures.
- FR-4.2: The agent must send this telemetry data to the backend every 5 seconds.

- FR-4.3: The dashboard must use Socket.io so graphs update live, without the page needing to be refreshed.

3.5 Compliance Scoring Engine

- FR-5.1: The system must check whether Windows Defender (antivirus) is switched on. If it is off, the risk score goes up by 20%.
- FR-5.2: The system must check the firewall status. If it is off, the risk score goes up by another 20%.
- FR-5.3: If USB storage is not restricted (the relevant registry value is not set to 4), the risk score increases by 15%.
- FR-5.4: The score must be worked out again every time new telemetry arrives, and grouped as follows: below 40 is LOW RISK (green), 40 to 69 is MEDIUM RISK (orange), and 70 or above is HIGH RISK (red).

3.6 Remote Execution Controls

- FR-6.1: Admins must be able to send a shutdown or reboot command from the live telemetry screen.
- FR-6.2: Commands must wait in a queue on the server until the target agent checks in and picks them up.
- FR-6.3: The agent must read the command and run the matching Windows instruction, either shutdown /s /t 0 (shutdown) or shutdown /r /t 0 (restart).

4. NON-FUNCTIONAL REQUIREMENTS

These requirements describe how well the system should perform, rather than exactly what it should do.

4.1 Performance

- Telemetry data must appear on the dashboard within 1 second of the server receiving it.
- Database searches for asset records must complete in under 300 milliseconds.
- The background agent must use less than 3% of CPU power and under 25 MB of memory on the monitored computer.

4.2 Security

- A database clean-up or log wipe must never delete the record for the agent that is currently active and connected.
- Every telemetry request to the API must include a valid JWT token in the request header (Authorization: Bearer <TOKEN>).
- User passwords must be salted and hashed with Bcrypt before they are stored in the database.

4.3 Reliability and Availability

- If the backend server's port is already in use, it must show a clear error and shut down safely instead of crashing without explanation.
- If a device loses its connection to the server, the agent must keep trying to reconnect automatically rather than stopping.

4.4 Maintainability and Portability

- The dashboard must look and behave the same way across major modern browsers.
- Server settings and API addresses must be stored in .env and config.json files, so they can be changed without editing the program's source code.

5. SYSTEM REQUIREMENTS

5.1 Hardware Requirements

Table 5.1 shows the recommended hardware for the server that runs the backend and database, and for the computers being monitored.

Component	Minimum Specification	Recommended Specification
Server machine	Quad-core CPU, 8 GB RAM, 50 GB SSD	Octa-core CPU, 16 GB RAM, 100 GB NVMe
Target machine (agent)	Dual-core CPU, 2 GB RAM, Windows 10/11	Quad-core CPU, 4 GB RAM, Windows 10/11

5.2 Software Requirements

Table 5.2 lists the software and libraries needed to run each part of the system.

Layer	Technologies
Frontend	React (Vite, JavaScript), Chart.js, React-icons, Axios, Socket.io-client
Backend	Node.js (v18+), Express, pg (PostgreSQL client), JWT, Bcrypt, Socket.io
Database	PostgreSQL (v14+)
EDR Agent	Python 3.10+ (packaged with PyInstaller), psutil, requests, winreg

5.3 Network Requirements

- Port 5000 must be open for the backend API and WebSocket traffic.
- Port 5173 must be open for the frontend development or preview server.
- A stable internal network connection is needed between every monitored device and the backend server, since the agent reports every 5 seconds.

6. FUTURE ENHANCEMENTS

The current system covers the core needs of asset tracking and security monitoring, but there is room to grow. Some ideas for future versions include:

1. Cross-Platform Agent Support: Extending the background agent so it can also check security settings on Linux (for example UFW or iptables rules) and macOS, not just Windows.
2. Notification Hub: Adding real-time email and SMS alerts for when a device's compliance score becomes critical, instead of relying only on the dashboard.
3. Mobile Companion App: Building a simple mobile app so administrators can check device status and respond to alerts while away from their desk.
4. Command Signing: Adding digital signatures to remote commands, so a restart or shutdown instruction cannot be faked or changed while travelling between the server and the agent.
5. Active Directory Integration: Connecting the system to an organization's existing Active Directory, so employee and device records can sync automatically instead of being entered by hand.
6. Historical Analytics: Adding longer-term reports and trend graphs, so admins can see how a device's health and compliance score has changed over weeks or months.

7. CONCLUSION

This document has laid out the full set of requirements for the Smart IT Asset Lifecycle & Security Compliance Management System, written in plain language so that developers, supervisors, and reviewers all share the same understanding.

By bringing together asset tracking, live security monitoring, and remote device control in one platform, the system aims to replace slow, manual record keeping with a faster, safer, and more connected way of managing IT equipment. The functional and non-functional requirements described here will guide the design, development, and testing stages that follow, and the future enhancements section outlines a clear path for improving the system further once the initial version is complete.